

附录 | FAQ

FAQ

需要什么基础？

建议具备编程能力，以及后端、数据工程、平台工程或算法工程中的任一基础。

可以跳着学吗？

可以，但建议至少先完成 Week01 和 Week02。

这套站点会持续更新吗？

会。前台会优先开放正式交付页面，后台生产材料不在前台暴露。

Week04 是否需要学生自己安装 Spark、Hive、Trino 或 Nessie？

不需要。Week04 的学生主路径是 `omnisupport-copilot` 仓库里的 Docker devbox：PostgreSQL 提供 SQL catalog，MinIO 提供 warehouse/object storage，Pylceberg 负责 catalog smoke、materialize、metadata inspection 和 demo report。

Week04 为什么优先用 devbox CLI，而不是直接从 Dagster 点按钮？

当前 Week04 的事实来源是 Pylceberg 命令链。Dagster 在这一周可以作为 thin wrapper 或后续编排入口，但在 Dagster 镜像完整拥有 Pylceberg 依赖之前，课程验收以 `runbooks/week04/README.md` 里的 devbox 命令和 `reports/week04/*` 证据为准。

如果课程站点和 OmniSupport Copilot 仓库命令不一致，以哪个为准？

以真实项目仓库为准。优先检查 `docs/blueprints/week04/course_site_sync_packet_v1.md` 和 `runbooks/week04/README.md`，课程站点只应该同步已经在仓库里落地的路径、命令和报告文件。

Week05 | Transform、语义层与受控 KPI 工具

为什么 Week5 不就是写几个 KPI SQL？

因为本周交付的是口径资产包：source、grain、tests、docs、lineage、registry、tool contract 和 audit 都要能被复核。SQL 只是其中一层实现。

dbt 是数据库吗？

不是。dbt 不存数据，也不画图。它把 SQL transform 组织成有依赖、有测试、有文档、有 artifacts 的工程。

staging、intermediate、marts 到底怎么分？

staging 负责稳定输入形状，intermediate 承接复杂业务组合，marts 面向下游消费。判断标准不是目录名，而是“一层该对谁负责”。

grain 为什么这么重要？

grain 是“一行代表什么”。如果 `support_case_mart`、`support_kpi_mart` 和 raw event 的 grain 混在一起，count、rate、avg 都会看起来能跑但业务含义错误。

data tests 和 unit tests 有什么区别？

data tests 断言模型结果或 source 假设，例如 not null、accepted values。unit tests 用小样本验证 SQL 逻辑边界，例如 first response 或 reopen 规则。

dbt docs generate 有什么实际用？

它不只是生成网页，还会产出 `manifest.json`、`catalog.json` 等 evidence，帮助字段解释、lineage、impact notes 和后续交付复盘。

Semantic Layer、MetricFlow 和本地 metric registry 是什么关系？

本地 registry 是 Week05 的最小语义契约；MetricFlow 和 dbt Semantic Layer 是未来可以迁移和扩展的方向。课程不把托管平台作为硬依赖。

为什么不把 dbt Cloud / 托管 Semantic Layer 作为学生硬依赖？

因为 Student Core 要能本地复现。托管平台能力可以作为未来路线，但不能替代本地 dbt Core、registry 和 tool contract 的可验证交付。

为什么 Agent 不能直接写 SQL？

Agent 裸写 SQL 会带来口径漂移、越权、非法 filter、成本失控、审计缺失和不可复现。本周用 `query_support_kpis_v1` 把查询收进白名单和审计边界。

为什么负例测试是工具验收的核心？

正例只能证明工具能查；负例能证明工具会拒绝不该发生的查询，例如未知指标、非法维度、角色越权和过大时间窗口。

为什么 mart 里有的指标不一定要开放给 Agent？

mart 中可计算，不代表业务定义、tests、docs、roles、负例和 audit 都准备好了。`avg_first_response_minutes` 这类指标可以先观察，暂不进 registry。

Week5 和 Week6 / Week8 / Week10 的边界是什么？

Week5 负责 analytics、metric registry 和安全指标查询工具 v1。Week6 继续做资产化编排和回填，Week8 消费指标与证据做检索一致性，Week10 扩展更完整的 tool/action/HITL 治理。

Week06 | 资产化数据工厂、回填与运行证据

Week06 是不是就是 Dagster UI 教程？

不是。Dagster UI 只是观察和触发 asset graph 的界面。Week06 真正要学的是：如何把脚本、表、指标和证据组织成可分区、可回填、可检查、可交接的数据资产系统。即使不用 UI，也要能通过 CLI、reports 和 runbook 复核同一条链路。

为什么脚本跑通还不够？

脚本跑通只能说明一次执行没有崩。数据工厂还要回答：跑的是哪个资产、哪个分区、上游输入是什么、检查是否通过、证据在哪里、下游能不能消费。如果这些问题没有结构化记录，团队只能依赖个人记忆，无法稳定交接。

asset 和 task 有什么区别？

task 强调动作，例如“跑一次 ingest”。asset 强调团队要维护和消费的数据对象，例如 `week06/silver/ticket_fact_partitioned`。Week06 关注 asset，因为下游关心的是数据状态、分区、质量和证据，而不是某个脚本有没有被调用过。

为什么 Student Core 默认 daily partition?

daily partition 足够小，便于课堂复现、定位缺口、做 dry-run backfill，也能连接 Week03 的 batch / incremental 思维。它不是生产环境唯一答案，但能让学员先掌握“按边界恢复”，而不是一失败就全量重跑。

backfill、replay、rerun、retry 到底怎么区分?

retry 是同一次执行里的瞬时重试；rerun 是重新跑同一个作业；replay 是用同一批输入重放当时语义；backfill 是补历史空洞或旧分区。Week06 要求先生成 dry-run plan，再决定动作，不靠感觉救火。

为什么 optional 依赖不能写 passed?

Week04 lakehouse 或 Week05 mart 如果没有真实跑通，只能写 `skipped` 或 `not_available`。写成 `passed` 会欺骗下游，导致 Week07 / Week08 / Week11 基于不存在的状态继续消费。诚实的不可用，比假的通过更安全。

run evidence 和日志有什么区别?

日志记录过程细节，通常给调试用。run evidence 是结构化交接证据，必须包含 asset、partition、status、reason code、report path 和 downstream decision。它的消费者不只是开发者，还包括助教、下游周次和后续治理。

为什么不在 Week06 直接上 OpenLineage / lakeFS?

因为 Week06 的 Student Core 是本地可复现的数据工厂 v1。OpenLineage、lakeFS、完整 lineage 平台是后续扩展方向。当前先把 asset graph、checks、evidence 和 runbook 做扎实，否则直接上平台只会放大混乱。

如果 Week04 / Week05 没完全跑通，Week06 怎么办?

把 Week04 / Week05 作为 optional dependency 观察，并在 evidence 中写清 `skipped` 或 `not_available`。不要为了让图变绿而假装它们通过。Week06 的核心路径仍然可以验证 definitions、backfill dry-run、checks 和 evidence schema。

Lab 里为什么先做 dry-run backfill，而不是直接写数据?

dry-run 让你先看清 partition、输入量、当前输出、gap reason、idempotency guard 和 downstream impact。直接写数据容易把“恢复演示”变成“破坏状态”。课堂默认先保留可复核计划，再由 runbook 决定是否进入真实写入。

Week07 | 非结构化数据工程、解析、切片与证据锚点

Week7 是不是 RAG 课?

不是。Week7 只做到 Week8-ready document asset: parse、section、chunk、anchor、quality gate。embedding、pgvector、hybrid retrieval、rerank 和 RAG API 属于 Week8。

为什么不能直接 PDF to text?

因为纯文本通常丢掉标题层级、表格结构、页码、bbox、文档版本和 source fingerprint。RAG 需要的是可回源证据，不只是可读文字。

Docling-first 是不是意味着 Unstructured 没价值?

不是。Docling-first 是 Student Core 的默认路线，保证本地可复现和结构 metadata；Unstructured 可以作为 fallback 或对照，但 fallback 必须写 capability warning。

为什么 OCR 默认不作为学生硬依赖？

OCR 会引入额外依赖、成本、扫描件质量和误识别风险。Student Core 先把非扫描文档和结构化 metadata 做稳，扫描件用 `ocr_disabled_in_student_core` 诚实记录限制。

`bbox_missing_reason` 为什么必须写？

因为缺 `bbox` 可能是正常不适用，也可能是 parser 丢信息。HTML / Markdown 可以是 not applicable；PDF `bbox` 缺失则至少要 warning，并写清原因。

为什么 citation 不能由 LLM 临时生成？

citation 是事实来源，不是文本生成结果。LLM 可以总结证据，但引用字段必须来自 Week7 生成的 evidence anchor。

section 和 chunk 有什么区别？

section 是文档结构单元，保留标题层级和位置；chunk 是 Week8 索引候选单元，由 section-aware 策略生成并继承 metadata。

fixed-size chunk 什么时候可以用？

可以作为 fallback 或对照，但不应该是 Week7 主路径。主路径应先尊重标题、表格、FAQ、步骤和页码边界，再控制 token budget。

为什么 chunk ID 不能用随机 UUID？

随机 ID 无法比较两次运行结果，也无法判断策略变更是否导致 chunk 漂移。Week7 需要稳定 ID 支撑回归和 bad case replay。

什么样的 chunk 不能进入 Week8？

空 chunk、孤儿 chunk、无 evidence anchor、缺 required metadata、PDF 无 page_no、疑似 PII 或 quarantine 的 chunk 都不能进入 Week8 索引。

`sample_shortfall` 是失败吗？

不是。它是诚实限制。样本不足时必须写当前样本数、缺口和扩展到 50+ 文档的抽样流程。

Week7 作业为什么要求 bad case review？

因为解析和切片策略只有通过坏案例才能改进。只写“全部通过”通常说明没有做认真复核。

如果 Week3 文档输入不完整怎么办？

可以使用 deterministic fallback，但必须写入 fallback reason，不能伪装成完整 raw object 路径已经成功。

Week7 和 Week8 的交接文件是什么？

核心是 `week8_ready_gate.json`、`chunk_quality_report.md`、sections/chunks/anchors 样例和 bad case review。Week8 只消费 gate 允许的 chunks。

Week09 | Agent Skills 开放标准与 Skill Pack

Week9 是不是教大家写提示词模板？

不是。Week9 交付的是 Skill Pack：把 Week2–Week8 的工程工艺封装成 Agent 可发现、可执行、可校验的工作流。提示词只是一小部分，真正的边界在 `SKILL.md`、scripts、references、manifest 和 validation report。

为什么 canonical source 放在 `skills/`，发现目标放在 `.agents/skills/`?

`skills/` 是随仓库版本管理的事实来源，便于 review、测试和发布；`.agents/skills/` 是本地 Agent 的发现入口。课程建议用复制脚本同步，而不是 symlink，避免不同工具和操作系统下行为不一致。

为什么要做 5 个最小 skills?

这 5 个 skills 对应 Week2–Week8 已经沉淀出的关键工程动作：契约检查、回填 runbook、RAG 契约检查、Prompt as Code 发布和 release check。它们足够小，才能被发现、执行和验证。

Skill 里可以直接执行修复动作吗?

默认不可以。Student Core 要求先 dry-run、生成报告、写 reason code，再由人或 runbook 决定是否执行破坏性动作。Week10 之后才会继续讨论更完整的 tool action 和 HITL 边界。

前序周资产没落地怎么办?

不要伪造。对应 skill 的 validation report 中写 `not_available` 或 `skipped`，并说明缺少哪个契约、报告或运行入口。诚实的不可用比假的通过更安全。